

Evidence-Bounded Structural Reproduction of a GPU sGS-HPR Solver for Large-Scale Multi-Period DC Optimal Power Flow

Thomas DiGregorio

Abstract—We assess whether the mathematical and computational route of a published GPU symmetric Gauss–Seidel Halpern-accelerated proximal-reflection (sGS-HPR) method for multi-period DC optimal power flow can be reconstructed and validated without the authors’ instances or code. The canonical LP, paper-order updates, stopping and KKT maps, structural equality solve, FP64 CPU oracle, and resident NVIDIA DGX Spark GPU path were implemented and checked. A sign inconsistency in the printed rank-one inverse is corrected; the corrected solve has maximum relative error 3.03×10^{-16} , whereas the printed sign has median relative error 0.206. Public MATPOWER reconstructions match all 18 published row and variable dimensions but none of the nonzero counts. Eleven rows were allocated and GPU-validated; ten also passed the required CPU track. The remaining CPU attempt reached the frozen 3,600-s limit. Three later rows stopped before allocation: T16 exceeded observed live memory budgets despite fitting the nominal 80% reference, T24 exceeded a conservative int32 planning envelope while its count-only nnz did not, and T32 exceeded int32 under both measures. Under a preregistered project-specific taxonomy, the result is D – Structural Reproduction. No paper-time reproduction or local speedup is claimed.

Index Terms—DC optimal power flow, Halpern iteration, proximal reflection, symmetric Gauss–Seidel decomposition, GPU sparse linear algebra, reproducibility, DGX Spark.

I. INTRODUCTION

Wang *et al.* propose a GPU sGS-HPR method for large-scale multi-period DC optimal power flow (DCOPF) and report results on an A100-SXM4-80GB GPU [1]. Reproducing that work requires more than running a related optimizer: the mathematical map, instance identity, implementation path, validation criteria, timing boundary, and hardware conditions must be separated. This study asks which of those layers can be recovered from the paper and public artifacts, and which conclusions remain unsupported.

The project froze an A–E decision framework before final synthesis: A exact, B near-exact, C mathematical, D structural, and E partial. This is a project-specific evidence-resolution taxonomy, not an ACM artifact badge or a replacement for the standard

repeatability/reproducibility/replicability terminology [13]. Structural reproduction means that the mathematical path, implementation structure, public-network scaling, validation, and resource behavior are reproduced while author-instance identity and controlled paper timing are not.

The principal result is **D – Structural Reproduction**. The mathematics and an FP64 GPU implementation are validated, but the author instances are unavailable, all 18 reconstructed sparse supports differ, and the frozen large-run campaign contains a required CPU timeout. A separately implemented validation checker records that the protocol and evidence are internally consistent; it does not convert the Stage 8 scientific decision from FAIL to PASS.

The contributions are:

- a self-contained canonical DCOPF and sGS-HPR implementation contract;
- independent projection, residual, physical, spectral, direct-solve, and CPU/GPU trajectory checks;
- a formal correction and stable implementation of the structural equality inverse;
- an 18-row structural ledger, an 11-row allocated benchmark record, and repeated timing distributions;
- fail-closed live-memory and sparse-index evidence with the planning and count-only interpretations kept distinct; and
- a public, versioned code/data release with machine-readable provenance and regeneration instructions.

The displayed source model has no outage or contingency index. This report therefore studies base-case multi-period DCOPF. N-1 SCOPF is new work and Stage 10 remains locked.

II. METHODS

A. Source paper, scope, and public data

The source paper is the 17-page article by Wang *et al.* [1]; the supplied PDF has SHA-256

7e9791646401e11bfdd9eb6bd94491ed0b592744581edd851ddb5e20dba4.

The paper applies HPR to the dual of a box-constrained LP, splits equality and inequality multipliers with an sGS sweep, and uses CSR sparse products on an A100. Its $O(1/k)$ KKT residual result follows the HPR framework [2].

Public base networks are MATPOWER 8.1 `case1354pegase`, `case2868rte`, and `case9241pegase`, pinned by upstream commit, Git blob, and SHA-256 [3], [4]. The PEGASE/RTE provenance is described in [5], [4]. The authors' renewable/storage placements, time series, reserve profiles, modified ramps, storage data, matrix construction, exact control code, and timer boundary were unavailable. Appendix A gives the deterministic replacement policy.

B. Mathematical formulation

The implementation contract is

$$\begin{aligned} \min_{x \in \mathbb{R}^n} \quad & c^\top x, \\ \text{s.t.} \quad & A_1 x = b_1, \\ & A_2 x \geq b_2, \\ & \ell \leq x \leq u, \end{aligned} \quad (1)$$

where $A = [A_1; A_2]$, $b = [b_1; b_2]$, and $C = [\ell, u]$. With $D = \mathbb{R}^{m_1} \times \mathbb{R}_+^{m_2}$, the dual is

$$\begin{aligned} \min_{y, z} \quad & -b^\top y + \delta_D(y) + \delta_C^*(-z), \\ \text{s.t.} \quad & A^\top y + z = c. \end{aligned} \quad (2)$$

Equality multipliers are free; inequality multipliers are nonnegative. The canonical inequality direction is always $A_2 x \geq b_2$.

The decision order is

$$x = (p_G, p_{RG}, p_{ESS}^{dc}, p_{ESS}^{ch}, r^u, r^d). \quad (3)$$

For T periods, N_L constrained lines, N_G conventional generators, N_{RG} renewables, and N_{ESS} storage devices,

$$n = T(3N_G + N_{RG} + 2N_{ESS}), \quad (4)$$

$$m_1 = T + N_{ESS}, \quad (5)$$

$$m_2 = 2TN_L + (4T - 2)N_G + 2TN_{ESS} + 2T. \quad (6)$$

Power balance and terminal storage energy form A_1 . Split branch flow, headroom/footroom, reserve, ramp, and storage-energy rows form A_2 ; remaining device limits form C .

The full KKT residual is

$$\mathcal{R}(y, z, x) = \begin{pmatrix} y - \Pi_D(y - Ax + b) \\ x - \Pi_C(x - z) \\ c - A^\top y - z \end{pmatrix}. \quad (7)$$

The paper's stopping rule separately normalizes primal feasibility, box normal-cone consistency, and stationarity. Its first stopping block is not the entire first block of (7); both maps are retained.

C. Implemented algorithm and sourced control policy

For $w = (y_1, y_2, z, x)$, the FP64 implementation follows the printed update order. The combined z /primal update gives

$$\bar{x}^{k+1} = \Pi_C[x^k + \sigma(A^\top y^k - c)]. \quad (8)$$

An equality solve computes $\bar{y}_1^{k+1/2}$ using y_2^k . Then

$$\bar{y}_2^{k+1} = \Pi_{\mathbb{R}_+^{m_2}}[y_2^k + \lambda^{-1}(b_2/\sigma - A_2 R_y)], \quad (9)$$

where λ safely bounds $\|A_2\|_2^2$. A second equality solve uses the new $\bar{y}_2^{k+1}$ to obtain $\bar{y}_1^{k+1}$. Finally,

$$\hat{w}^{k+1} = 2\bar{w}^{k+1} - w^k, \quad (10)$$

$$w^{k+1} = \frac{1}{k+2}w^0 + \frac{k+1}{k+2}\hat{w}^{k+1}. \quad (11)$$

The anchor w^0 is fixed.

Production preprocessing applies ten Ruiz equilibration passes [6], one simultaneous Pock–Chambolle diagonal step with $\alpha = 1$ [7], and complete-vector b/c normalization. Initial $\sigma = 1$. The DCOPF paper does not publish exact restart or adaptive-penalty code; Appendix B states the transferred HPR-LP rules, formulas, guards, and source pin explicitly [2]. This is a sourced reconstruction, not author-code identity.

D. CPU, GPU, and reference solvers

The CPU oracle uses SciPy sparse operators in FP64, zero initial state, both equality sweeps, original-coordinate recovery, and independent residual and physical checks. SciPy/HiGHS supplies the reference objective via `linprog(method='highs-ds')` with presolve, 10^{-9} primal and dual feasibility tolerances, and a 3,600-s limit [8], [9].

The GPU path uses CuPy 14.1.1 arrays, FP64 CSR matrices, resident state, and reused workspaces [10]. A low-level descriptor records `CUSPARSE_SPMV_CSR_ALG2`; normal and transpose probes verify the selected path. NVIDIA documents CSR ALG2 as deterministic for non-transpose SpMV [11]. Transfer ledgers reject unexpected full-state copies inside the iteration loop.

E. Acceptance and timing design

Every accepted candidate must meet normalized stopping blocks $\leq 5 \times 10^{-5}$, raw KKT $\leq 10^{-2}$, physical violation $\leq 10^{-2}$ MW/MWh, and scaled objective gap to HiGHS $\leq 2 \times 10^{-4}$. Stage 7–8 requires HiGHS, CPU FP64, and GPU FP64 unless a later continuation explicitly skips CPU without rewriting the original decision.

Each successful timing track has one correctness solve, one warm-up, and at least five measured solves. Relative range above 0.2 escalates the track to nine repeats. The report retains median, minimum, maximum, standard deviation, and interquartile range (IQR). HiGHS time includes the SciPy interface/model setup and solve; CPU/GPU times use separately prepared solver workspaces and include their solver-specific iteration/residual boundary. These definitions are not interchangeable. The T6 CPU event is one censored correctness attempt, not a timing median.

TABLE I
STAGE MAP AND IRREVERSIBLE ACCEPTANCE BOUNDARY.

Stage	Scope	Principal evidence	Decision
0	Paper audit	identity, equations, ambiguities, unavailable artifacts	Pass
1	Generic LP	projections, residuals, toy LP oracles	Pass
2	DCOPF build	MATPOWER parsing, PTDF, dimensions, physical checks	Pass
3	CPU sGS-HPR	paper-order fixed- σ solver and trajectories	Pass
4	Structural solve	direct oracle, sign resolution, conditioning	Pass
5	Controls	reversible scaling, restart, adaptive σ	Pass
6	DGX backend	resident FP64 path, ALG2, CPU/GPU parity, timing ledger	Pass
7	Benchmarks	six allocated small/medium public reconstructions	Pass
8	Large campaign	four passes, T6 CPU timeout, three resource classifications	Fail
9	Synthesis	frozen taxonomy, report, index, PDF, separate checker	D

III. VERIFICATION

A. Staged validation map

Table I prevents a later aggregate score from erasing an earlier failed gate. Checker totals were 120/120 for Stages 0–7, 12/12 for the strict Stage 8 protocol, and 13/13 for the approved GPU-only continuation. Those totals verify evidence handling; the Stage 8 decision remains FAIL.

B. Structural equality correction

Lemma 1 (stable inverse of the reduced equality block). Let $D_2 \succ 0$ be diagonal, $a > 0$, d a vector, and $\alpha = d^\top D_2^{-1} d$. For

$$\begin{bmatrix} aI_T & \mathbf{1}d^\top \\ d\mathbf{1}^\top & D_2 \end{bmatrix} \begin{bmatrix} y_{11} \\ y_{12} \end{bmatrix} = \begin{bmatrix} r_{11} \\ r_{12} \end{bmatrix}, \quad (12)$$

define

$$\hat{r} = r_{11} - \mathbf{1}d^\top D_2^{-1} r_{12}, \quad (13)$$

$$\gamma = a - T\alpha, \quad \mu = T^{-1}\mathbf{1}^\top \hat{r}. \quad (14)$$

If $\gamma > 0$, the unique solution is

$$y_{11} = \frac{\hat{r} - \mu\mathbf{1}}{a} + \frac{\mu}{\gamma}\mathbf{1}, \quad (15)$$

$$y_{12} = D_2^{-1}(r_{12} - d\mathbf{1}^\top y_{11}). \quad (16)$$

Proof. Eliminating y_{12} yields $(aI_T - \alpha\mathbf{1}\mathbf{1}^\top)y_{11} = \hat{r}$. The zero-mean subspace has eigenvalue a and $\text{span}\{\mathbf{1}\}$ has eigenvalue $a - T\alpha = \gamma$. Decomposing $\hat{r}$ into mean and zero-mean components gives (15); back-substitution gives (16). $\square$

The printed expression uses the sign pattern of a plus rank-one update and fails a direct linear-system oracle. Equation (15) avoids forming the cancellation-prone Woodbury denominator in the solve. The code also evaluates γ through a positive-sum identity (Appendix C). On the sign fixture, the printed formula’s median relative error is 0.206; the corrected maximum relative error is 3.03×10^{-16} . Across condition numbers from 1 to 25,616, the largest normalized residual is 2.93×10^{-14} .

C. Numerical, physical, and trajectory checks

Analytic LPs verify box and nonnegative projections, (7), the separate stopping blocks, and the combined z /primal identity. Dense eigendecomposition is the small-case authority for λ ; sparse `eigsh` and seeded power

TABLE II
FROZEN GATES AND LARGEST ACCEPTED OBSERVATIONS.

Gate	Threshold	Maximum	Result
Normalized primal	5e−5	2.017e−8	Pass
Normalized stationarity	5e−5	9.659e−6	Pass
Normalized box	5e−5	8.831e−13	Pass
Raw KKT	0.01	0.009635	Pass
Physical violation	0.01	0.006311	Pass
Objective gap	2e−4	4.285e−8	Pass
Per-solve deadline	3600 s	3600.093 s	T6 CPU fail

iteration cross-check it. PTDF flows are compared with an independent angle solve, including phase-shift offsets. CPU/GPU checks compare intermediate trajectories, policy events, final objectives, and recovered physical states.

IV. RESULTS

A. Structural reconciliation

All 18 Table II (m, n) pairs match. Zero reconstructed nnz counts match; differences range from approximately −8.14% to −36.66%. Thus the public models reproduce the dimension algebra but not the author sparse supports. Paper/local timing ratios are invalid before hardware differences are considered.

B. Allocated benchmarks

The GPU completed and validated all 11 allocated rows; CPU completed 10. The T6 GPU result does not override the preregistered all-track requirement.

No speedup is claimed from Fig. 1. The shared-axis overlay makes the magnitude and scaling patterns visible without making the clocks commensurate. The elevated CPU maxima on several nine-repeat tracks show why a median alone is insufficient. The five GPU samples are tightly clustered on the large rows, but this is local repeatability evidence, not an A100 comparison.

C. Resource boundaries

DGX Spark has 128 GB LPDDR5x unified memory [12]. The run-time system reported 130,663,165,952 bytes (121.690 GiB). The preregistered guard did not allocate against that nominal total. It required the projected host assembly plus device plan to fit within both 80% of observed host-available pages and 80% of observed CUDA-free bytes.

TABLE III

ALLOCATED STAGE 7–8 RECONSTRUCTIONS. TIMES ARE LOCAL SOLVER-CORE MEDIAN AFTER CORRECTNESS AND WARM-UP; MEASURED DISPERSION IS IN APPENDIX D. A DASH DENOTES NO ACCEPTED TIMING SAMPLE.

Stage	Reconstruction	m	n	$\text{nnz}(A)$	HiGHS (s)	CPU (s)	GPU (s)	Result
7	case1354 T4	20,192	4,208	4,799,808	1.463	13.190	1.013	Pass
7	case1354 T16	82,124	16,832	19,228,464	7.265	47.399	2.924	Pass
7	case1354 T48	247,276	50,496	57,896,368	32.095	153.046	9.481	Pass
7	case1354 T96	495,004	100,992	116,420,464	101.242	336.657	21.084	Pass
7	case2868 T4	40,163	9,488	19,073,056	5.366	60.350	3.939	Pass
7	case2868 T16	163,823	37,952	76,354,336	22.138	250.601	15.408	Pass
8	case2868 T48	493,583	113,856	229,507,104	76.239	804.863	49.968	Pass
8	case2868 T64	658,463	151,808	306,303,136	108.232	1,078.892	68.383	Pass
8	case2868 T96	988,223	227,712	460,334,496	163.593	1,621.905	105.022	Pass
8	case9241 T4	152,774	24,700	342,863,272	142.479	3,087.218	193.632	Pass
8	case9241 T6	230,376	37,050	514,308,838	963.957	–	357.544	Fail

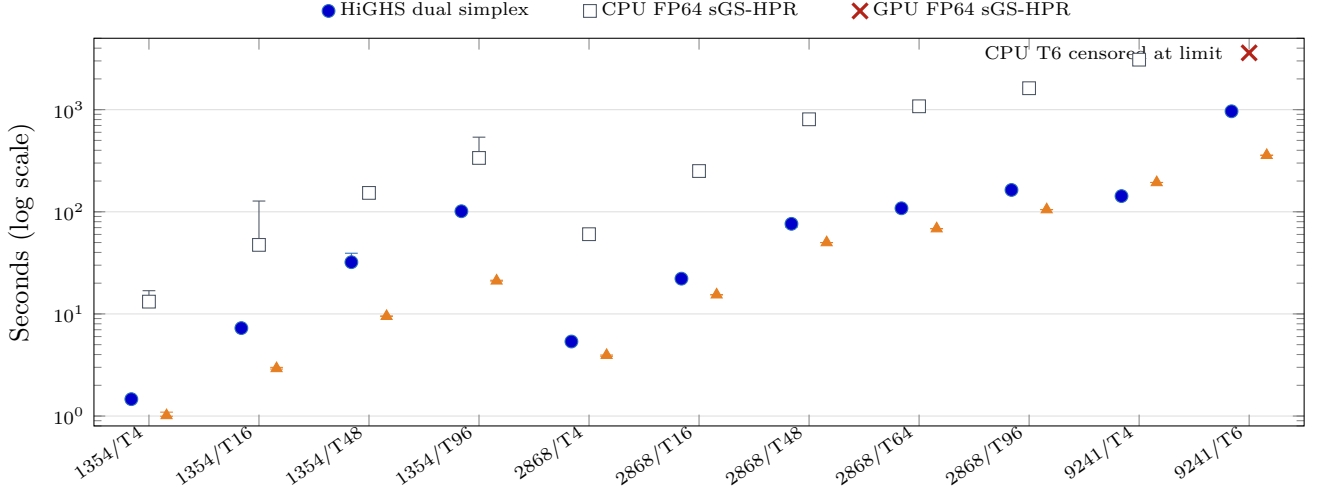


Fig. 1. Combined local timing evidence on one shared logarithmic axis. Each method is horizontally offset within a reconstruction; markers are medians and whiskers span measured minima and maxima. CPU T6 is one censored solver boundary and algorithms differ. The overlay aids visual comparison but does not establish a speedup because solver boundaries and algorithms differ.

TABLE IV
INTERPRETATION OF FINAL STAGE 8 PREALLOCATION OUTCOMES.

Row	Status	Projected GiB	Planning nnz	Exact count nnz	int32 max	Interpretation
T16	memory blocked	94.435	1,687,613,820	1,371,647,068	2,147,483,647	Above live budgets 65.784/65.496 GiB, but below the 97.352-GiB nominal 80% reference.
T24	index blocked	141.663	2,531,600,260	2,057,650,132	2,147,483,647	Conservative planning envelope exceeds int32; count-only reconstructed nnz remains below it.
T32	index blocked	188.897	3,375,704,460	2,743,770,956	2,147,483,647	Both planning envelope and count-only reconstructed nnz exceed int32.

The maximum cumulative process-lifetime host high-water mark was 95.375 GiB at T6; it is not an isolated solve peak. CUDA memory observations are snapshots, not per-solve high-water marks.

V. DISCUSSION

A. What was and was not reproduced

The canonical signs, variables, constraint families, all 18 (m, n) pairs, update order, Halpern weights, tolerance, Ruiz/Pock-Chambolle preprocessing, initial σ , policy cadence, FP64 CSR path, and requested ALG2 selection are reproduced. CPU/GPU trajectories and all accepted original-space checks agree.

Author-instance identity is not reproduced. Placement, time series, reserve and ramp edits, storage parameters, costs, PTDF reference/sparsification, control code, source revision, precision details, and timer boundary are missing. Equation (55) and Table II imply $m_1 = T + N_{ESS}$ while Appendix A prose implies $T(1 + N_{ESS})$; the rank-one sign is inconsistent with the Schur complement; and the security-constrained label lacks an explicit outage model.

B. Classification and uncertainty

Class A is rejected because author artifacts, hardware, and exact outputs are unavailable. B is rejected because every nnz differs and timing is uncontrolled. C is achieved

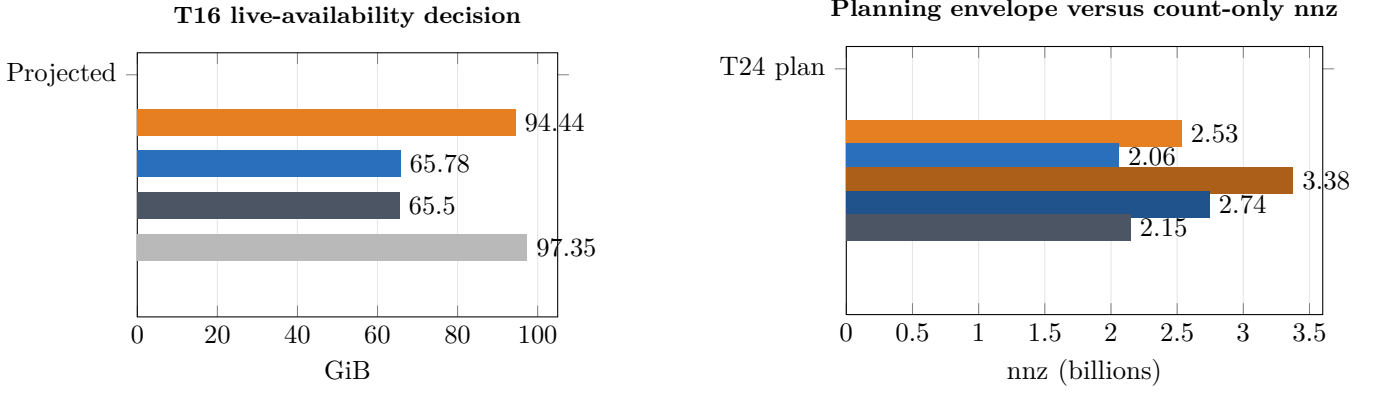


Fig. 2. Fail-closed resource evidence. T16 is a live-availability safety block, not a nominal 128-GB ceiling. T24 is a conservative policy-envelope block, not a demonstrated materialized overflow. T32 exceeds signed-int32 under both the envelope and count-only reconstruction. No row reached LP construction, solver allocation, or an OOM crash.

mathematically but understates the executed public-network scaling. E understates the validated solver and complete structural ledger. Under the frozen internal taxonomy, D is the narrowest defensible label.

The major uncertainties are instance construction and external performance, not whether the local code can execute. Timing cannot disentangle algorithm, support, implementation, hardware, runtime, or stopwatch effects. The T6 CPU track is censored and gives no median. T16/T24/T32 give safe classifications, not solve times. Memory APIs limit peak attribution. The adaptive policy is a pinned HPR-LP transfer rather than unpublished DCOPF author code.

The most valuable follow-up is author-instance reconciliation: obtain modified cases, profiles, placements, parameters, construction code, precision, control logic, and timer rules; first match (m, n, nnz), then objective/KKT fingerprints, then an equivalent A100 boundary. If those artifacts remain unavailable, 64-bit sparse indices, partitioned operators, or lower-peak assembly would be a new scalability study rather than a correction to this result.

VI. CONCLUSION

This study reconstructs and validates the paper’s path from canonical DCOPF to a resident FP64 GPU sGS-HPR solver. It records a quantitatively material rank-one sign error, separates stopping from full KKT validation, verifies CPU/GPU trajectories, retains repeated timing dispersion, and distinguishes three different preallocation outcomes. The evidence also contains a genuine required-track timeout and complete sparse-support mismatch. The result is therefore **D – Structural Reproduction**—stronger than a component demonstration and narrower than an exact, near-exact, or controlled performance reproduction.

APPENDIX A

DETERMINISTIC PUBLIC-INSTANCE CONSTRUCTION

The three canonical case-file SHA-256 values are 1b08b25a...b89 (1354), 2b30e894...861 (2868), and

593a58ec...5f3 (9241); complete values are in `configs/benchmarks/stage_7_small_medium.json`.

APPENDIX B

PREPROCESSING, RESTART, PENALTY, AND TIMING DEFINITIONS

The sGS multiplier metric for $\Delta y = (\Delta y_1, \Delta y_2)$ is

$$v = A_1^\top \Delta y_1 + A_2^\top \Delta y_2, \quad (17)$$

$$Q_y(\Delta y) = (A_1 v)^\top (A_1 A_1^\top)^{-1} (A_1 v) + \lambda \|\Delta y_2\|^2. \quad (18)$$

The restart merit is

$$\tilde{R}^2 = \|\Delta x\|^2 / \sigma + 2(A\Delta x)^\top \Delta y + \sigma Q_y(\Delta y). \quad (19)$$

Policy checks occur every 100 iterations; the first checkpoint forces a restart. Later checkpoints restart for sufficient decay $\tilde{R} \leq 0.2\tilde{R}_{ref}$, necessary decay with no local progress $\tilde{R} \leq 0.6\tilde{R}_{ref}$ and $\tilde{R} > \tilde{R}_{prev}$, or a long inner loop $k_{inner} \geq 0.2k_{total}$. An accepted restart copies the proximal point to anchor and current state and resets the inner counter.

With reference and candidate states,

$$\Delta_x = \|x_c - x_r\|, \quad \Delta_y = \sqrt{Q_y(y_c - y_r)}, \quad (20)$$

$$\sigma_{new} = \Delta_x / \Delta_y. \quad (21)$$

Both movements must lie strictly in $(10^{-16}, 10^{12})$, and the normalized dual/primal infeasibility ratio in $(10^{-8}, 10^8)$. Guard failure resets σ to 1. The policy source is HPR-LP commit 1941fbcfbf2dae14e4a439b22f0ea1e1c05f4a29 [2].

HiGS core time is the SciPy `linprog` call, including interface/model setup and solve. CPU/GPU sGS-HPR times begin after their solver-specific model/preconditioner/workspace preparation and include the iterative solve and scheduled residual work. CUDA initialization, model construction, preprocessing, first-run compilation, allocation, transfers, solver initialization, iteration, nested residuals, recovery, and complete runner wall time are separately named in `results/tables/stage_9_timing_decomposition.csv`. Nested residual time is part of loop time and must not be added again.

TABLE V
FROZEN PUBLIC-INSTANCE CONSTRUCTION RECIPE (SEED 20260803).

Component	Frozen rule
Networks	MATPOWER 8.1 at commit <code>1a828c7af590714499284e36ee9c81273388c594</code> ; three case blobs and SHA-256 values are recorded in the configuration.
Load	Flat profile; active withdrawal is MATPOWER $P_d + G_s$ each hour.
Generators	Retain every row; originally offline rows are fixed to zero output and reserve; ramp limit is $0.1P_{\max}/h$.
Renewables	Cycle through generator buses in generator-row order; aggregate nameplate is 10% of base load, equally divided; availability 1; minimum 0.
Storage	Every second renewable location, cycling as needed; aggregate power 1% of base load, equal division; 4-h duration; initial state 50%; $\eta^{ch} = \eta^{dc} = 0.95$.
Reserve	Up and down requirements are each 1% of base load.
Costs	MATPOWER linear and constant polynomial terms; reject nonzero higher-order terms; renewable and storage-loss penalties are \$1/MWh.
Branches	Preserve positive rateA; zero rateA receives an outward-rounded finite bound proven redundant over the variable box; inactive rows are kept as zero-flow count rows but excluded from topology.
PTDF	Public type-3 reference bus; RHS chunks of 128; zero tolerance 10^{-12} ; phase-shift affine offsets retained; angle limits omitted because they are absent from the source model.
Time	One-hour intervals; no timing-driven tuning after the contract freeze.

APPENDIX C ALGEBRAIC STABILITY AND CONDITIONING

The positive-sum implementation of the reduced eigenvalue is

$$\gamma = G + R + \sum_s \frac{(\eta_s^{ch} - 1/\eta_s^{dc})^2}{(\eta_s^{ch})^2 + 1/(\eta_s^{dc})^2}, \quad (22)$$

which avoids subtracting nearly equal a and $T\alpha$. Nonpositive or scale-small γ is rejected.

APPENDIX D TIMING DISPERSION

The CSV retains standard deviations and repeat counts in addition to the statistics shown here. Five repeats were planned; high-variability tracks were escalated to nine without deleting earlier samples.

APPENDIX E REPRODUCIBILITY, CODE, AND DATA AVAILABILITY

Code and data availability. Source, public case snapshots, configurations, immutable raw evidence, generated tables/figures, the machine-readable index, and this PDF are available at <https://github.com/thomas-digregorio/sGS-HPR-DC-SCOPF>. The report release is the Git tag `stage9-report-v3`; the benchmark-evidence source base is `c08c53b7ef5d2bde006728c76fb43fe621685e20`, and the first complete Stage 9 synthesis is `85007e9e752ea5e082bd0266cf43393fc8f3e7e2`. The public network source is MATPOWER 8.1, DOI 10.5281/zenodo.15871662 [3].

Missing source information. Exact reproduction would require the authors’ modified cases, placements, profiles, reserve and ramp data, storage parameters, matrix construction, PTDF policy, precision, control code, source revision, and timer definition. Unknown values were not guessed or fitted to published timings.

Machine-readable evidence. `results/stage_9_result_index.json` records paper identity, classification, stage decisions, environment, thresholds, evidence

hashes, table/figure hashes, timing semantics, resource interpretations, and the Stage 10 lock. The complete structural ledger is `results/tables/stage_9_structural_reconciliation.csv`; the benchmark and dispersion fields are in `results/tables/stage_9_benchmarks.csv`.

Regeneration. From the repository root:

```
python scripts/generate_stage_9_artifacts.py
python scripts/generate_stage_9_artifacts.py --check
python scripts/check_stage_9.py --output results/raw/stage_9/
stage_9_checks.json
python -m pytest -q
python -m ruff check .
python -m ruff format --check .
```

The full command inventory is `docs/regeneration_commands.md`. Report regeneration does not rerun accepted or failed DGX benchmark allocations.

REFERENCES

- [1] Q. Wang, G. Zhang, Y. Yang, C. Ren, W. Wu, X. Zhao, M. Skoglund, and D. Sun, “An efficient GPU-based Halpern accelerating algorithm for large-scale DC optimal power flow,” *IEEE Trans. Power Syst.*, vol. 41, no. 3, pp. 2187–2204, May 2026, doi: 10.1109/TPWRS.2025.3635652.
- [2] K. Chen, D. Sun, Y. Yuan, G. Zhang, and X. Zhao, “HPR-LP: An implementation of an HPR method for solving linear programming,” *Math. Program. Comput.*, vol. 18, pp. 183–210, 2026, doi: 10.1007/s12532-025-00292-0.
- [3] R. D. Zimmerman and C. E. Murillo-Sánchez, *MATPOWER*, version 8.1, 2025, doi: 10.5281/zenodo.15871662; and R. D. Zimmerman, C. E. Murillo-Sánchez, and R. J. Thomas, “MATPOWER: Steady-state operations, planning, and analysis tools for power systems research and education,” *IEEE Trans. Power Syst.*, vol. 26, no. 1, pp. 12–19, 2011.
- [4] C. Jozs, S. Fliscounakis, J. Maeght, and P. Panciatici, “AC power flow data in MATPOWER and QCQP format: iTesla, RTE snapshots, and PEGASE,” arXiv:1603.01533, 2016.
- [5] S. Fliscounakis, P. Panciatici, F. Capitanescu, and L. Wehenkel, “Contingency ranking with respect to overloads in very large power systems taking into account uncertainty, preventive, and corrective actions,” *IEEE Trans. Power Syst.*, vol. 28, no. 4, pp. 4909–4917, 2013, doi: 10.1109/TPWRS.2013.2251015.
- [6] D. Ruiz, “A scaling algorithm to equilibrate both rows and columns norms in matrices,” Rutherford Appleton Laboratory, Tech. Rep. RAL-TR-2001-034, 2001.
- [7] T. Pock and A. Chambolle, “Diagonal preconditioning for first order primal-dual algorithms in convex optimization,” in *Proc. IEEE ICCV*, 2011, pp. 1762–1769, doi: 10.1109/ICCV.2011.6126441.

TABLE VI
STRUCTURAL-SOLVE CONDITIONING AGAINST THE DIRECT FP64 ORACLE.

Fixture	Rows	κ_2	Max relative error	Max normalized residual	Max component error
No storage, T1	1	1.00	2.22e−16	1.67e−16	2.91e−11
No storage, T17	17	1.00	0	0	0
Ideal storage, T1	2	86.01	2.18e−16	6.66e−16	3.73e−9
One storage, T2	3	3.58	3.03e−16	3.21e−16	1.16e−10
Extreme efficiency, T32	33	25,616.42	3.00e−15	2.93e−14	1.14e−9
Heterogeneous storage, T5	9	548.59	1.15e−15	1.05e−15	3.11e−10
Many ideal devices, T16	48	292.03	1.53e−14	6.44e−15	4.42e−9

TABLE VII
MEDIAN [MINIMUM, MAXIMUM] SECONDS; PARENTHESIS GIVES IQR. T6 CPU IS ONE CENSORED CORRECTNESS ATTEMPT AND IS NOT SUMMARIZED AS A DISTRIBUTION.

Reconstruction	HiGHS	CPU FP64 sGS-HPR	GPU FP64 sGS-HPR
1354/T4	1.463 [1.453,1.480] (0.015)	13.190 [11.836,16.879] (1.571)	1.013 [0.995,1.092] (0.010)
1354/T16	7.265 [7.258,7.283] (0.005)	47.399 [46.635,127.424] (3.906)	2.924 [2.885,2.996] (0.097)
1354/T48	32.095 [31.249,39.290] (1.651)	153.046 [152.641,155.015] (1.680)	9.481 [9.476,9.531] (0.048)
1354/T96	101.242 [100.940,101.613] (0.176)	336.657 [330.488,537.835] (7.599)	21.084 [20.947,21.380] (0.274)
2868/T4	5.366 [5.340,5.375] (0.026)	60.350 [60.018,60.807] (0.253)	3.939 [3.846,3.959] (0.098)
2868/T16	22.138 [22.082,22.421] (0.219)	250.601 [249.273,255.187] (1.645)	15.408 [15.394,15.413] (0.006)
2868/T48	76.239 [70.892,77.949] (4.411)	804.863 [801.569,808.401] (3.749)	49.968 [49.927,49.998] (0.027)
2868/T64	108.232 [107.335,108.458] (0.598)	1,078.892 [1,077.122,1,135.954] (38.376)	68.383 [68.292,68.392] (0.008)
2868/T96	163.593 [158.374,164.829] (1.394)	1,621.905 [1,594.076,1,634.230] (19.300)	105.022 [105.001,105.174] (0.032)
9241/T4	142.479 [141.937,142.810] (0.376)	3,087.218 [3,070.085,3,096.832] (13.350)	193.632 [193.603,193.741] (0.120)
9241/T6	963.957 [960.497,966.056] (1.420)	3,600.093 censored correctness attempt	357.544 [357.462,357.751] (0.206)

- [8] P. Virtanen *et al.*, “SciPy 1.0: Fundamental algorithms for scientific computing in Python,” *Nature Methods*, vol. 17, pp. 261–272, 2020, doi: 10.1038/s41592-019-0686-2.
- [9] Q. Huangfu and J. A. J. Hall, “Parallelizing the dual revised simplex method,” *Math. Program. Comput.*, vol. 10, pp. 119–142, 2018.
- [10] R. Okuta, Y. Unno, D. Nishino, S. Hido, and C. Loomis, “CuPy: A NumPy-compatible library for NVIDIA GPU calculations,” in *Proc. Workshop on Machine Learning Systems, NeurIPS*, 2017.
- [11] NVIDIA Corp., “cuSPARSE library,” CUDA Toolkit Documentation, 2026. [Online]. Available: <https://docs.nvidia.com/cuda/cusparses/>
- [12] NVIDIA Corp., “DGX Spark User Guide: Hardware overview,” 2026. [Online]. Available: <https://docs.nvidia.com/dgx/dgx-spark/hardware.html>
- [13] Association for Computing Machinery, “Artifact review and badging—current,” version 1.1, 2020. [Online]. Available: <https://www.acm.org/publications/policies/artifact-review-and-badging-current>